Verilog 第三次实验报告

实验名称	8 位算术逻辑运算模块 (ALU) 的设计				
学生姓名	赵宇哲	学号	2411094	指导老师	杨飞
实验地点	公共实验楼 A 区 306 教室		实验时间	2025-10-15 星期三第三四节	

1 实验项目名称

基于 Vivado 的加法器设计与仿真

2 实验平台

Nandgame 在线平台

Xilinx Vivado Design Suite 2017.1（64-bit）

3 实验内容

3.1 Nandgame 实现 Switching

3.1.1 选择器

s 作为选择比特，要实现对电路的控制。由真值表可知，s 为 0 时，d_0 控制输入；s 为 1 时，d_1 控制输入。我们可以通过 inv+and 的组合实现 s 对支路开闭的控制。

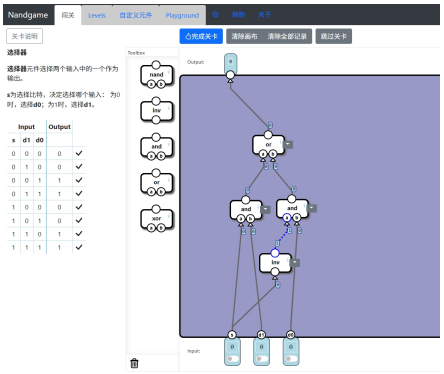


图 1: 选择器

3.1.2 开关

本题与“选择器”相反，是由 s 控制输出端。由真值表可知，s 为 0 时，从 c_0 输出；s 为 1 时，从 c_1 输出。但 s 端直接与 and-c_1 相连时，只要 s 为 0，c_1 输出便一定为 0，而 and-c_2 上由于由 inv，在此时恰好可由输入端 d 决定输出内容。

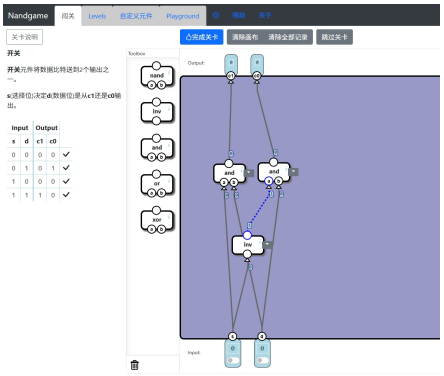


图 2: 开关

3.2 Nandgame 实现 Arithmetic Logic Unit

3.2.1 Logic Unit

我们可以认为 op1 是“更高一层”的决定信号，即：op=0 时，有 op0=0 输出 X and Y，op0=1 输出 X or Y（“次一层”决定信号 op0）；op=1 时，有 op0=0 输出 X xor Y，op0=1 输出 invert X。从而形成下图的两层结构。

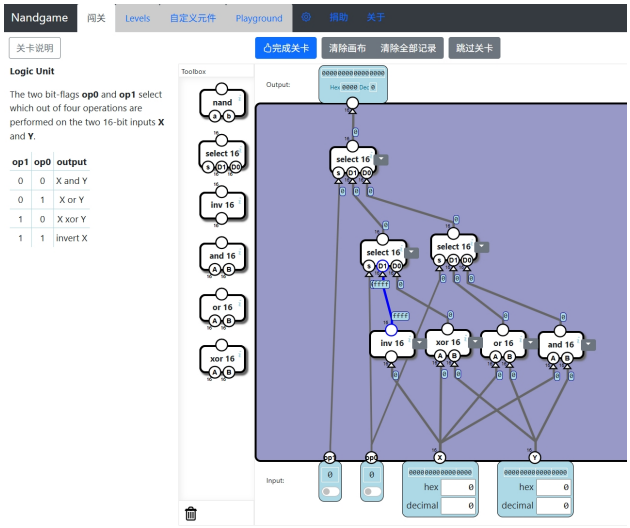


图 3: Logic Unit

3.2.2 Arithmetic Unit

思路与 Logic Unit 几乎相同，只是需要考虑 X、Y 的顺序问题，同样形成两层结构。

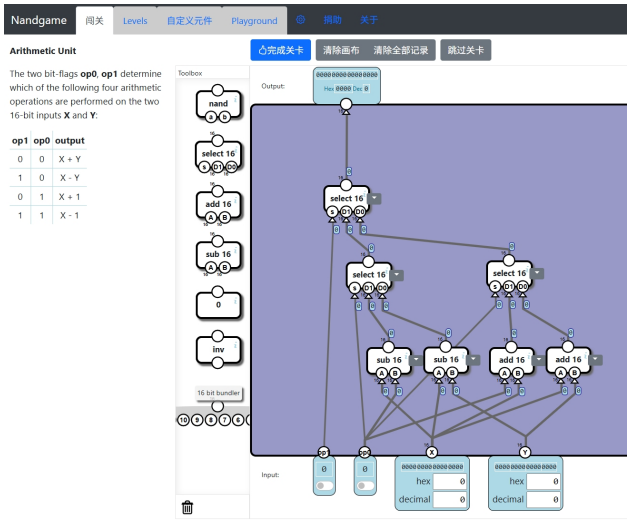


图 4: Arithmetic Unit

3.2.3 ALU

大框架与 Logic Unit 和 Arithmetic Unit 保持一致，只是在输入端新增了一个二层结构用于调整输入。通过观察真值表，**对 X 端**，我们认为 zx 是这一层次结构的“更高一层”的决定信号，即：zx=1 时，X 输出 0；zx=0 时，sw=0，输出 X，sw=1，输出 Y（下一层次），**对 Y 端**，输出只由 sw 控制，sw=0 时，输出 Y；sw=1 时，输出 X。

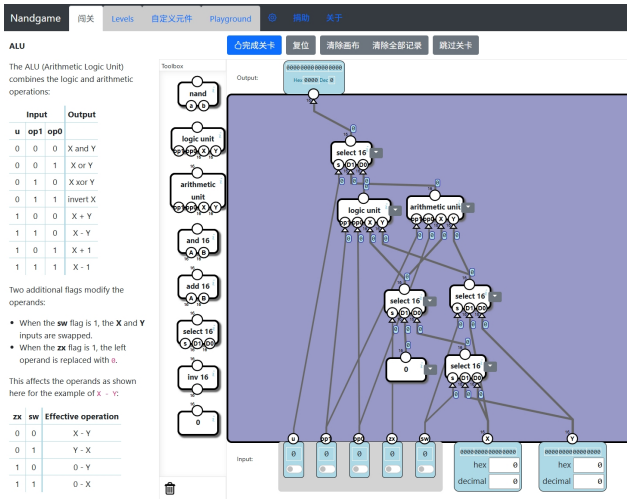


图 5: ALU

3.2.4 条件

这道题可以将电路划分为左右两部分：左侧是三种判别条件（`lt`、`eq`、`gt`），右侧是实际输入情况（`condition`）。由此，电路的输出逻辑可简化为：

$$\text{Output} = (lt \wedge \text{condition}_{lt}) \vee (eq \wedge \text{condition}_{eq}) \vee (gt \wedge \text{condition}_{gt})$$

即可得到下图所示的电路。

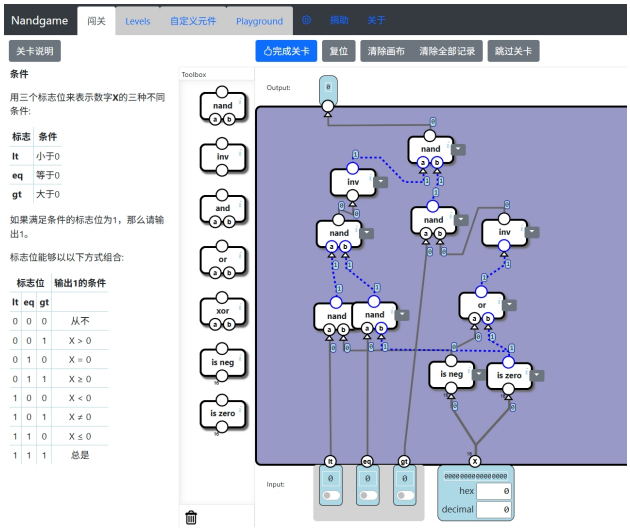


图 6: 条件

3.3 Vivado 实现 ALU

ALU 代码与仿真

3.3.1 设计代码

```
module simple_alu (
    // 输入端口
    input  [7:0] ina,
    input  [7:0] inb,

    // 输出端口
    output [7:0] sumab,
    output       sumflag,
    output [7:0] leftshiftA,
    output       lessflag,
    output       equalflag,
    output       bitXorflag
);

    // a和b相加
    assign {sumflag, sumab} = ina + inb;

    // a左移b位
    assign leftshiftA = ina << inb;

    // a < b
    assign lessflag = (ina < inb) ? 1'b1 : 1'b0;

    // a == b
    assign equalflag = (ina == inb) ? 1'b1 : 1'b0;

    // a的缩减异或（所有位进行异或，即奇偶校验）
    assign bitXorflag = ^ina;

endmodule
```

3.3.2 仿真代码

```
`timescale 1ns / 1ps

module tb_simple_alu;

    // 声明用于连接模块输入的寄存器(reg)
    reg [7:0] test_ina;
    reg [7:0] test_inb;

    // 声明用于连接模块输出的线网(wire)
    wire [7:0] w_sumab;
    wire       w_sumflag;
    wire [7:0] w_leftshiftA;
    wire       w_lessflag;
    wire       w_equalflag;
    wire       w_bitXorflag;


    // 实例化
    simple_alu uut (
        .ina(test_ina),
        .inb(test_inb),
        .sumab(w_sumab),
        .sumflag(w_sumflag),
        .leftshiftA(w_leftshiftA),
        .lessflag(w_lessflag),
        .equalflag(w_equalflag),
        .bitXorflag(w_bitXorflag)
    );

    // 初始块，用于生成测试激励信号
    initial begin

        // --- 测试案例 1: a < b ---
        // ina = 10 (00001010, 2个1 -> 偶数)，期望 bitXorflag = 0
        test_ina = 8'd10;
        test_inb = 8'd20;
        #10;

        // --- 测试案例 2: a = b ---
        // ina = 55 (00110111, 5个1 -> 奇数)，期望 bitXorflag = 1
        test_ina = 8'd55;
        test_inb = 8'd55;
        #10;

        // --- 测试案例 3: a > b ---
        // ina = 250 (11111010, 6个1 -> 偶数)，期望 bitXorflag = 0
        test_ina = 8'd250;
        test_inb = 8'd15;
        #10;

        // --- 测试案例 4: 左移操作 ---
        // ina = 13 (00001101, 3个1 -> 奇数)，期望 bitXorflag = 1
        test_ina = 8'b00001101;
        test_inb = 8'd3;
        #10;

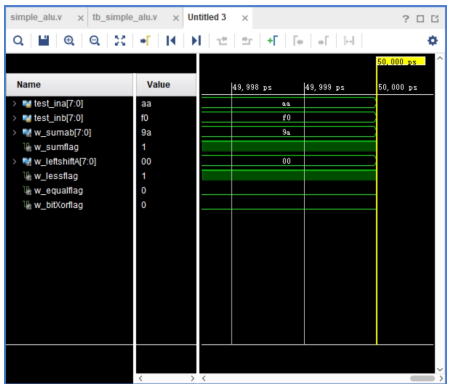
        // --- 测试案例 5: 缩减异或 ---
        // ina = 0xaa (10101010, 4个1 -> 偶数)，期望 bitXorflag = 0
        test_ina = 8'b10101010;
        test_inb = 8'b11110000;
        #10;

        $finish;
    end

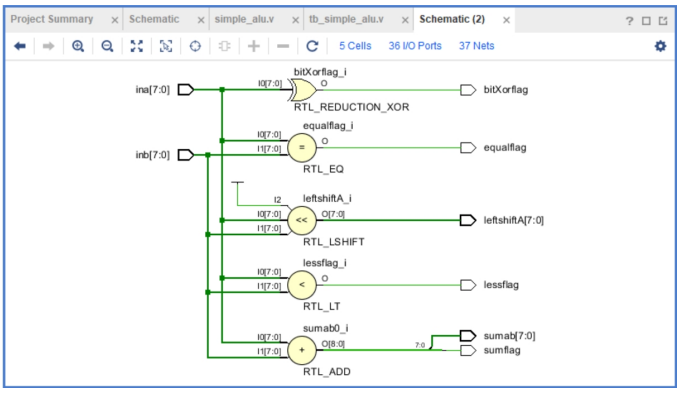
endmodule
```

ALU 代码与仿真

3.3.3 仿真结果



ALU 波形图



ALU 电路

4 收获总结

通过 **Switching** 和 **Arithmetic Logic Unit** 两部分实验，我了解到某些信号可以作为 **控制信号**，既可以控制有效输入端，也可以控制输出端。同时，多个控制信号会形成 **层级结构**，从而实现更复杂的多层控制逻辑，以完成更加多样的功能。

此外，最后一个 **Nandgame** 实验“条件”给我带来了很大的启发，让我更加直观地理解了数字电路在实际中的应用场景。例如，在该实验中所实现的检测电路，就可以作为检测电路，可以应用于黑盒等具体场景。

总用时没刻意记录，大概将近一个小时。